kafka

You are viewing documentation for an older version (2.5) of Kafka. For up-to-date documentation, see the latest version.

Quick Start

Tags: Kafka Docs

Step 1: Get Kafka

Download the latest Kafka release and extract it:

```
$ tar -xzf kafka_2.12-2.5.0.tgz
$ cd kafka_2.12-2.5.0
```

Step 2: Start the Kafka environment

NOTE: Your local environment must have Java 8+ installed.

Run the following commands in order to start all services in the correct order:

```
# Start the ZooKeeper service
# Note: Soon, ZooKeeper will no longer be required by Apache Kafka.
$ bin/zookeeper-server-start.sh config/zookeeper.properties
```

Open another terminal session and run:

```
# Start the Kafka broker service
$ bin/kafka-server-start.sh config/server.properties
```

Once all services have successfully launched, you will have a basic Kafka environment running and ready to use.

Step 3: Create a topic to store your events

Kafka is a distributed *event streaming platform* that lets you read, write, store, and process *events* (also called *records* or *messages* in the documentation) across many machines.

Example events are payment transactions, geolocation updates from mobile phones, shipping orders, sensor measurements from IoT devices or medical equipment, and much more. These events are organized and stored in *topics*. Very simplified, a topic is similar to a folder in a filesystem, and the events are the files in that folder.

So before you can write your first events, you must create a topic. Open another terminal session and run:

```
$ bin/kafka-topics.sh --create --topic quickstart-events --bootstrap-server localhost
```

All of Kafka's command line tools have additional options: run the `kafka-topics.sh` command without any arguments to display usage information. For example, it can also show you details such as the partition count of the new topic:

```
$ bin/kafka-topics.sh --describe --topic quickstart-events --bootstrap-server localhc
Topic:quickstart-events  PartitionCount:1    ReplicationFactor:1 Configs:
    Topic: quickstart-events Partition: 0    Leader: 0   Replicas: 0 Isr: 0
```

Step 4: Write some events into the topic

A Kafka client communicates with the Kafka brokers via the network for writing (or reading) events. Once received, the brokers will store the events in a durable and fault-tolerant manner for as long as you need—even forever.

Run the console producer client to write a few events into your topic. By default, each line you enter will result in a separate event being written to the topic.

```
$ bin/kafka-console-producer.sh --topic quickstart-events --bootstrap-server localhos
This is my first event
This is my second event
```

You can stop the producer client with `Ctrl-C` at any time.

Step 5: Read the events

Open another terminal session and run the console consumer client to read the events you just created:

```
$ bin/kafka-console-consumer.sh --topic quickstart-events --from-beginning --bootstra
This is my first event
This is my second event
```

You can stop the consumer client with `Ctrl-C` at any time.

Feel free to experiment: for example, switch back to your producer terminal (previous step) to write additional events, and see how the events immediately show up in your consumer terminal.

Because events are durably stored in Kafka, they can be read as many times and by as many consumers as you want. You can easily verify this by opening yet another terminal session and re-running the previous command again.

Step 6: Import/export your data as streams of events with Kafka Connect

You probably have lots of data in existing systems like relational databases or traditional messaging systems, along with many applications that already use these systems. Kafka Connect allows you to continuously ingest data from external systems into Kafka, and vice versa. It is thus very easy to integrate existing systems with Kafka. To make this process even easier, there are hundreds of such connectors readily available.

Take a look at the Kafka Connect section learn more about how to continuously import/export your data into and out of Kafka.

Step 7: Process your events with Kafka Streams

Once your data is stored in Kafka as events, you can process the data with the Kafka Streams client library for Java/Scala. It allows you to implement mission-critical real-time applications and microservices, where the input and/or output data is stored in Kafka topics. Kafka Streams combines the simplicity of writing and deploying standard Java and Scala applications on the client side with the benefits of Kafka's server-side cluster technology to make these applications highly scalable, elastic, fault-tolerant, and

distributed. The library supports exactly-once processing, stateful operations and aggregations, windowing, joins, processing based on event-time, and much more.

To give you a first taste, here's how one would implement the popular `WordCount` algorithm:

```
KStream<String, String> textLines = builder.stream("quickstart-events");

KTable<String, Long> wordCounts = textLines
            .flatMapValues(line -> Arrays.asList(line.toLowerCase().split(" ")))
            .groupBy((keyIgnored, word) -> word)
            .count();

wordCounts.toStream().to("output-topic"), Produced.with(Serdes.String(), Serdes.Long(
```

The Kafka Streams demo and the app development tutorial demonstrate how to code and run such a streaming application from start to finish.

Step 8: Terminate the Kafka environment

Now that you reached the end of the quickstart, feel free to tear down the Kafka environment—or continue playing around.

1. Stop the producer and consumer clients with `Ctrl-C`, if you haven't done so already.
2. Stop the Kafka broker with `Ctrl-C`.
3. Lastly, stop the ZooKeeper server with `Ctrl-C`.

If you also want to delete any data of your local Kafka environment including any events you have created along the way, run the command:

```
$ rm -rf /tmp/kafka-logs /tmp/zookeeper
```

Congratulations!

You have successfully finished the Apache Kafka quickstart.

To learn more, we suggest the following next steps:

- Read through the brief Introduction to learn how Kafka works at a high level, its main concepts, and how it compares to other technologies. To understand Kafka in more detail, head over to the Documentation.

- Browse through the Use Cases to learn how other users in our world-wide community are getting value out of Kafka.
- Join a local Kafka meetup group and watch talks from Kafka Summit, the main conference of the Kafka community.

Last modified December 19, 2025: Add license info to every source file and handle `documentation.html` links for all versioned paths. (#764) (379dba2230)